

Informe de Auditoría del Sistema Web

Integrantes del grupo:

Jorge Illera Rivera

Iker Argulo Galán

Oscar Lijerón Pérez

Bruno Izaguirre Martínez de Marañón

María Bogajo

Ander Javier Corral

Fecha:

09/11/2024

Índice de Contenidos

1	Introducción	3
2	Metodología	3
3	Vulnerabilidades	3
3.1	Rotura de control de acceso	3
3.1.1	Soluciones implementadas	3
3.2	Fallos criptográficos	3
3.2.1	Soluciones implementadas	3
3.3	Inyección	3
3.3.1	Cross-Site Scripting (XSS)	3
3.3.2	Soluciones implementadas	4
3.4	Diseño inseguro	5
3.4.1	Soluciones implementadas	5
3.5	Configuración de seguridad insuficiente	5
3.5.1	Soluciones implementadas	5
3.6	Componentes vulnerables y obsoletos	5
3.6.1	Soluciones implementadas	5
3.7	Fallos de identificación y autenticación	5
3.7.1	Soluciones implementadas	5
3.8	Fallos en la integridad de datos y software	5
3.8.1	Soluciones implementadas	5
3.9	Fallos en la monitorización de la seguridad	5
3.9.1	Soluciones implementadas	5
4	Conclusiones	5
5	Bibliografía	5
6	Apéndices	5

Índice de Figuras

1	Errores de inyección detectados por ZAP.	3
---	--	---

Índice de Tablas

1 Introducción

Este informe detalla la auditoría de seguridad del sistema web utilizando OWASP ZAP, con el objetivo de identificar y mitigar vulnerabilidades.

2 Metodología

La auditoría se realizó mediante un análisis de ZAP, ejecutando...

3 Vulnerabilidades

3.1 Rotura de control de acceso

3.1.1 Soluciones implementadas

3.2 Fallos criptográficos

3.2.1 Soluciones implementadas

3.3 Inyección

En la sección de inyección se ha identificado una amenaza de alto riesgo. La vulnerabilidad detectada corresponde a un ataque de Cross-Site Scripting (XSS), como se puede observar en la Figura 1.



Figura 1: Errores de inyección detectados por ZAP.

3.3.1 Cross-Site Scripting (XSS)

La vulnerabilidad CWE-79 corresponde a Cross-Site Scripting (XSS) Reflejado, que permite a un atacante inyectar código malicioso en los parámetros de entrada del sistema, como los campos de formularios o las URL. Cuando un usuario accede a una URL o un formulario que contiene este código inyectado, el navegador del usuario ejecuta dicho código, lo que puede llevar a la manipulación de datos, el robo de cookies, la ejecución de scripts no autorizados, y otros ataques de seguridad.

Causa de la vulnerabilidad

La vulnerabilidad XSS detectada se originaba en el archivo `modify_item.php`, más concretamente en el campo de la fecha de lanzamiento y en otros campos de entrada en el formulario de modificación de un juego. Esto se debía a que los valores de entrada eran reflejados en la respuesta sin la sanitización o validación adecuada, permitiendo a los atacantes inyectar y ejecutar código JavaScript en el navegador del usuario.

Pruebas y comportamiento observado

Sin la sanitización de entrada: OWASP ZAP detectó que ciertos campos permitían la inyección de código JavaScript en los valores de los formularios.

Por ejemplo, en el campo de la fecha, se podía ingresar un valor como

`<script>alert(1);</script>`, lo cual provocaba la ejecución de un alert en el navegador.

Conclusiones

La vulnerabilidad de XSS reflejada fue una amenaza crítica para la aplicación, ya que podía permitir a los atacantes ejecutar código malicioso en el navegador de los usuarios. Gracias a la implementación de sanitización y validación estrictas en los campos de entrada y salida, esta vulnerabilidad se ha mitigado de manera efectiva. La aplicación ahora cumple con un estándar de seguridad más alto, protegiendo los datos y la experiencia de los usuarios frente a posibles ataques de inyección de código.

3.3.2 Soluciones implementadas

Para mitigar la vulnerabilidad de XSS y otros posibles ataques de inyección, se implementaron las siguientes soluciones:

- **Sanitización de entrada y escapado de salida:**

Se utilizó la función `htmlspecialchars()` en PHP para escapar caracteres especiales (como `¡, ¿, &, ", ')` en todas las salidas. Esto asegura que el contenido reflejado en el navegador sea tratado como texto y no como código ejecutable.

- **Validación y sanitización de fecha de lanzamiento:**

Se implementó una expresión regular (`preg_match`) para asegurarse de que las fechas ingresadas cumplan con el formato `YYYY-MM-DD`.

- **Sanitización de datos entrantes:**

Para cada valor de entrada recibido a través del formulario, se aplicó `mysqli_real_escape_string()` para evitar que cualquier código especial o malicioso llegue a la base de datos.

- **Validación de Campos Numéricos (rating y price):**

Los valores de `rating` y `price` se convirtieron a tipo `float` usando `floatval()` para asegurarse de que se almacenen y procesan correctamente como valores numéricos.

Resultados de la Segunda Auditoría

Después de implementar las soluciones descritas, se realizó un segundo análisis con OWASP ZAP. Los resultados fueron positivos: no se detectaron vulnerabilidades de XSS ni otros problemas de inyección.

3.4 Diseño inseguro

3.4.1 Soluciones implementadas

3.5 Configuración de seguridad insuficiente

3.5.1 Soluciones implementadas

3.6 Componentes vulnerables y obsoletos

3.6.1 Soluciones implementadas

3.7 Fallos de identificación y autenticación

3.7.1 Soluciones implementadas

3.8 Fallos en la integridad de datos y software

3.8.1 Soluciones implementadas

3.9 Fallos en la monitorización de la seguridad

3.9.1 Soluciones implementadas

4 Conclusiones

5 Bibliografía

References

- [1] OWASP, "Cross-Site Scripting (XSS)", <https://owasp.org/www-community/attacks/xss/>,
accedido el 3 de noviembre de 2024.

6 Apéndices